

IT112 Web System and Technologies
Phase 4: Form Submission and Data Insertion
Lab Report by SURVIVORS

Group Members:

Jexson Sedon
John Roan Ballester
Michelle Diaz
Judy Pearl Balictar
Alyssa Jenille Reantaso
Ronel Garcia

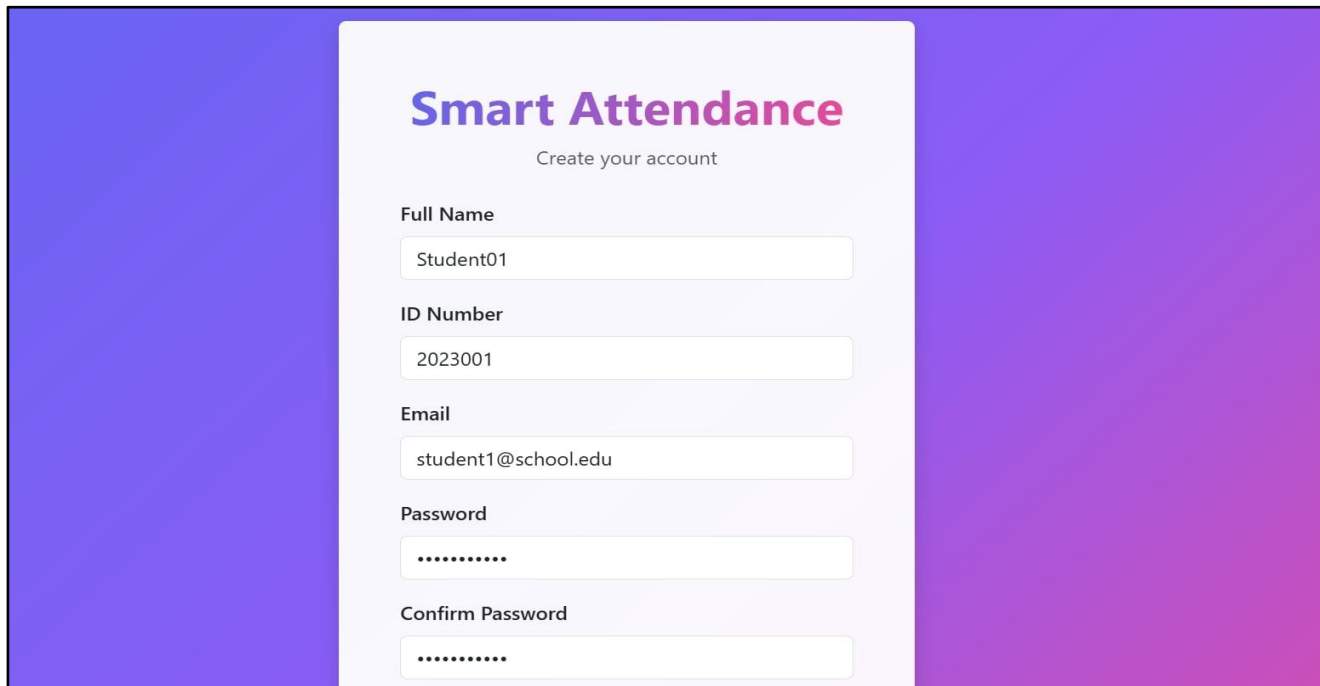
Submitted to:

Guillermo V. Red,Jr,DIT
Assistant Professor IV



I. Screenshot of HTML Form Interface

a. Student



The screenshot shows a web form titled "Smart Attendance" with the subtitle "Create your account". The form is set against a purple gradient background. It contains five input fields: "Full Name" (with "Student01"), "ID Number" (with "2023001"), "Email" (with "student1@school.edu"), "Password" (with masked dots), and "Confirm Password" (with masked dots).

Smart Attendance
Create your account

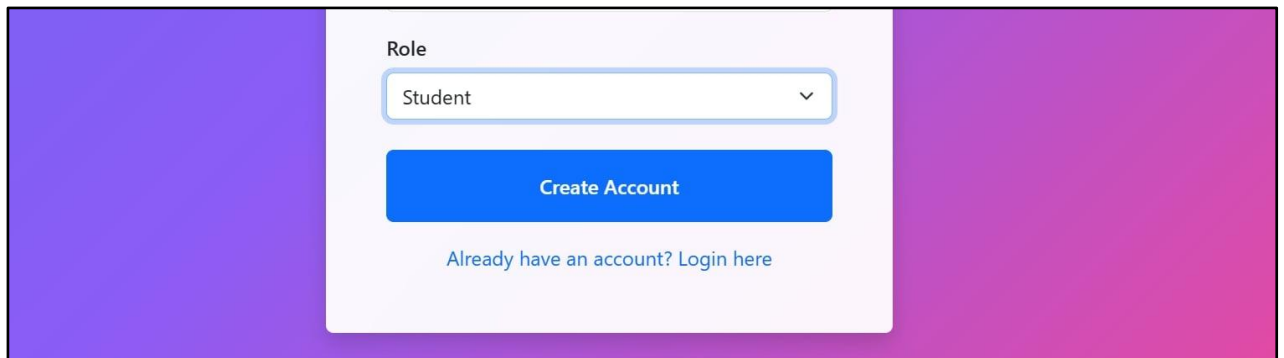
Full Name

ID Number

Email

Password

Confirm Password



The screenshot shows the bottom portion of the "Smart Attendance" account creation form. It features a "Role" dropdown menu with "Student" selected, a blue "Create Account" button, and a link "Already have an account? Login here".

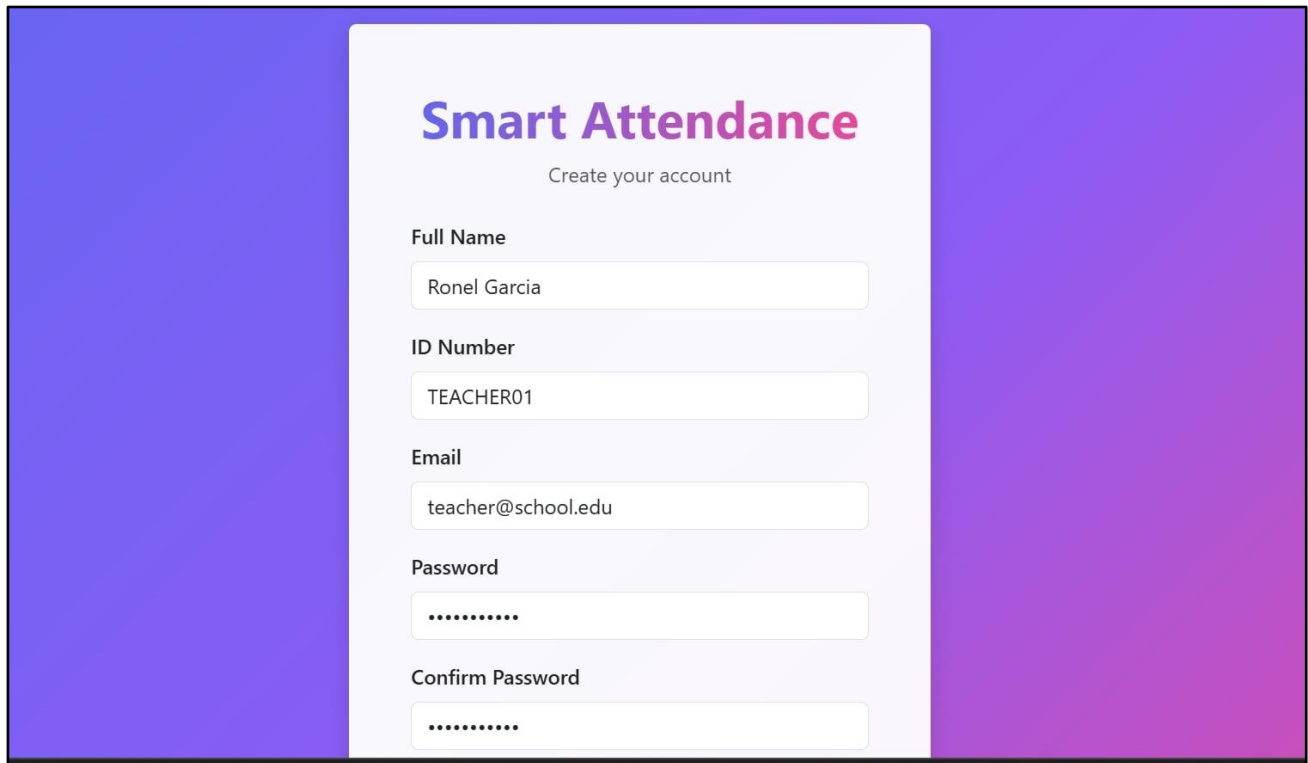
Role

[Create Account](#)

[Already have an account? Login here](#)



b. Teacher



The screenshot shows a registration form titled "Smart Attendance" with the subtitle "Create your account". The form is set against a purple-to-pink gradient background. It contains five input fields: "Full Name" (filled with "Ronel Garcia"), "ID Number" (filled with "TEACHER01"), "Email" (filled with "teacher@school.edu"), "Password" (filled with seven dots), and "Confirm Password" (filled with seven dots).

Smart Attendance
Create your account

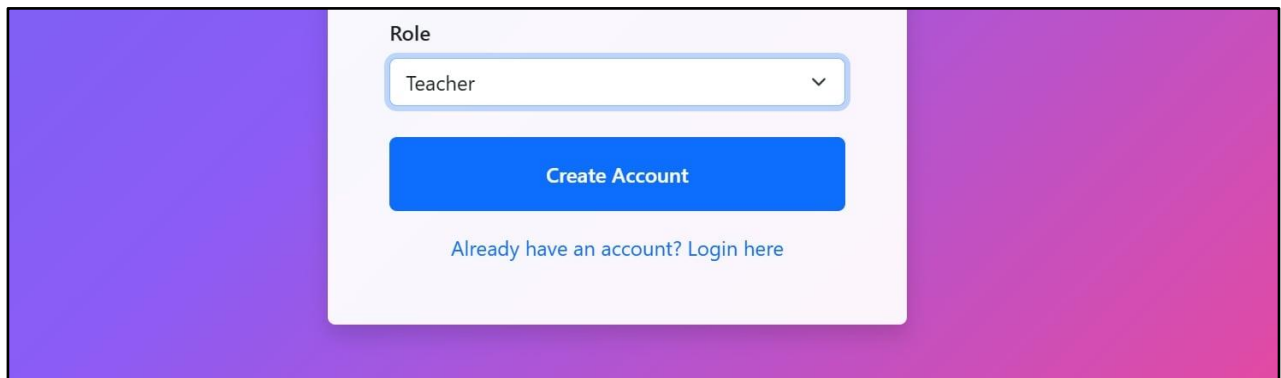
Full Name
Ronel Garcia

ID Number
TEACHER01

Email
teacher@school.edu

Password
.....

Confirm Password
.....



This screenshot shows the bottom portion of the registration form. It features a "Role" dropdown menu with "Teacher" selected, a blue "Create Account" button, and a link that says "Already have an account? Login here".

Role
Teacher

Create Account

[Already have an account? Login here](#)

Notes:

This screenshot shows the frontend registration form used for collecting user information such as name, ID number, email, and password. The form is connected to the backend API using JavaScript `fetch()`.



II. Screenshot of JavaScript API Connection Code

a. Api.js

```
1  const API_CONFIG = {
2    BASE_URL: 'http://localhost:3000/api',
3
4    async apiCall(endpoint, method = 'GET', body = null, token = null) {
5      const url = `${this.BASE_URL}${endpoint}`;
6      const options = {
7        method,
8        headers: {
9          'Content-Type': 'application/json'
10       }
11     };
12
13     if (token) {
14       options.headers['Authorization'] = `Bearer ${token}`;
15     }
16
17     if (body) {
18       options.body = JSON.stringify(body);
19     }
20
21     try {
22       const response = await fetch(url, options);
23       const data = await response.json();
24
25       if (!response.ok) {
26         if (response.status === 401) {
27           localStorage.removeItem('token');
28           localStorage.removeItem('currentUser');
29           window.location.href = 'login.html';
```

```
29       window.location.href = 'login.html';
30       throw new Error('Session expired. Please login again.');
```

```
31     }
32     throw new Error(data.message || 'API request failed');
33   }
34
35   return data;
36 } catch (error) {
37   console.error('API Error:', error);
38   throw error;
39 }
40 }
41 };
42
43 if (typeof module !== 'undefined' && module.exports) {
44   module.exports = API_CONFIG;
45 }
```



b. Auth.js

```
1 const AUTH = {
2   isLoggedIn() {
3     const token = localStorage.getItem('token');
4     const user = localStorage.getItem('currentUser');
5     return !!token && user;
6   },
7
8   getCurrentUser() {
9     try {
10      return JSON.parse(localStorage.getItem('currentUser'));
11    } catch {
12      return null;
13    }
14  },
15
16  getToken() {
17    return localStorage.getItem('token');
18  },
19
20  logout() {
21    localStorage.removeItem('token');
22    localStorage.removeItem('currentUser');
23    window.location.href = 'login.html';
24  },
25
26  requireAuth() {
27    if (!this.isLoggedIn()) {
28      window.location.href = 'login.html';
29      return false;
30    }
31  }
32}
```

```
29   return false;
30 }
31 return true;
32 },
33
34 redirectIfLoggedIn() {
35   if (this.isLoggedIn()) {
36     const user = this.getCurrentUser();
37     if (user.role === 'student') {
38       window.location.href = 'student-dashboard.html';
39     } else if (user.role === 'teacher') {
40       window.location.href = 'teacher-dashboard.html';
41     } else if (user.role === 'admin') {
42       window.location.href = 'admin-dashboard.html';
43     }
44   }
45 }
46
47 (method) setupLogoutButtons(): void
48
49 setupLogoutButtons() {
50   document.querySelectorAll('.logout-btn, #logoutBtn').forEach(btn => {
51     btn.addEventListener('click', (e) => {
52       e.preventDefault();
53       this.logout();
54     });
55   });
56 }
```

```
54   },
55
56   setupUserInfo() {
57     const user = this.getCurrentUser();
58     if (user) {
59       document.querySelectorAll('.user-name').forEach(el => {
60         el.textContent = user.username || 'User';
61       });
62       document.querySelectorAll('.user-role').forEach(el => {
63         el.textContent = user.role || 'Student';
64       });
65     }
66   }
67 };
68
69 document.addEventListener('DOMContentLoaded', () => {
70   const currentPage = window.location.pathname.split('/').pop();
71   if (['login.html', 'register.html', 'index.html', ''].includes(currentPage)) {
72     AUTH.redirectIfLoggedIn();
73   }
74 });
```

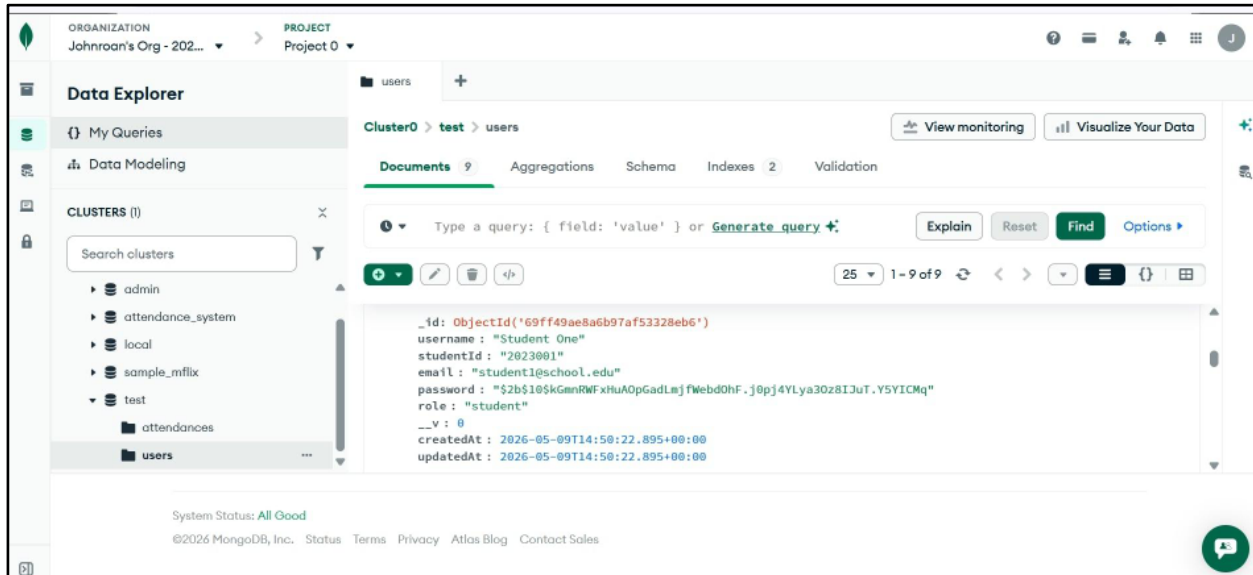
Notes:

This screenshot shows the JavaScript code responsible for handling form submission and sending POST requests to the backend API using `fetch()`. It also includes error handling and success alerts.

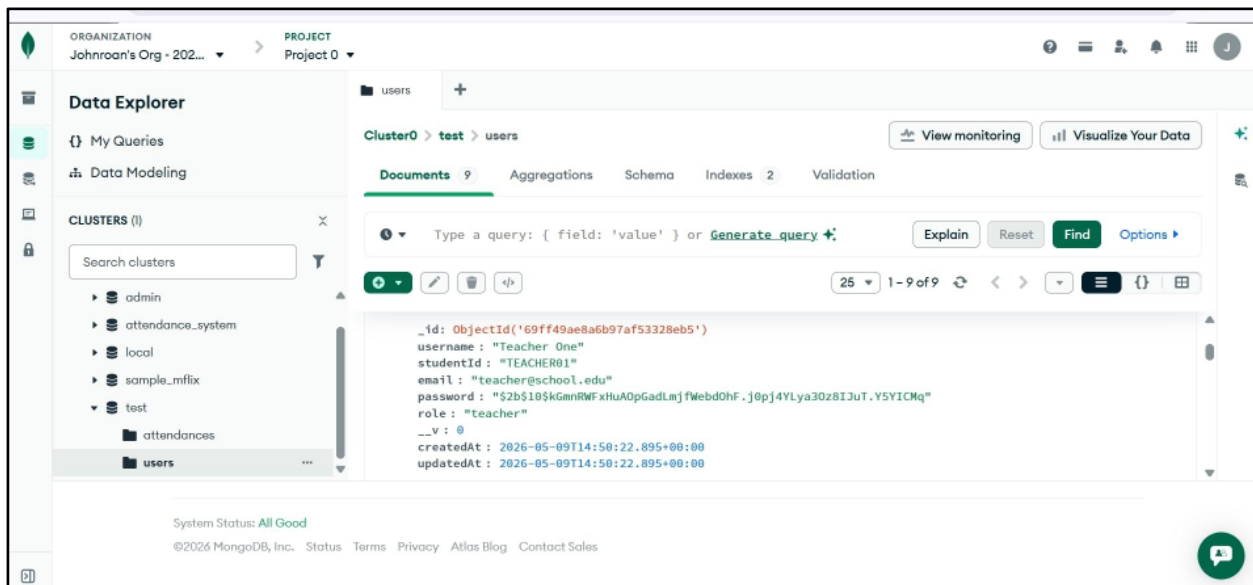


III. Screenshot of MongoDB Inserted Document

a. Student



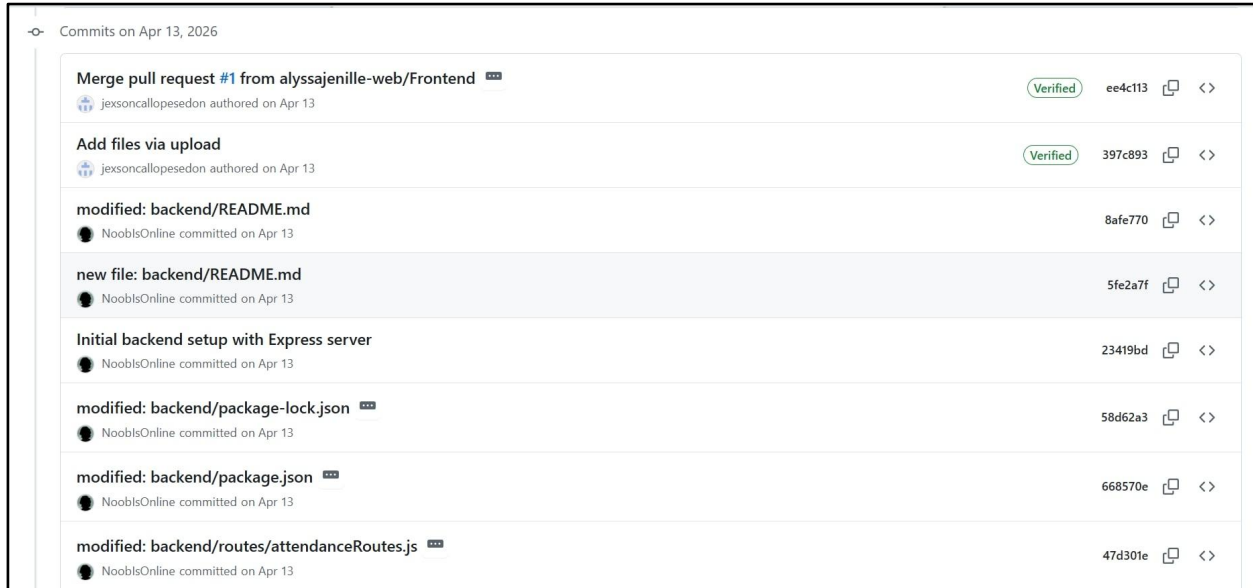
b. Teacher



Notes:

This screenshot confirms that the submitted form data was successfully stored in the MongoDB Atlas database collection after the API request was processed.

IV. Screenshot of GitHub Commits



Notes:

This screenshot displays the backend commits pushed to GitHub. Proper commit messages were used to document the progress and updates of the project.

V. Short Reflections of Each Member

Project Manager (Jexson Sedon)

This phase improved our teamwork and coordination skills. Managing the project workflow helped us complete the tasks efficiently.

Frontend Manager (Judy Pearl Balictar)

I learned how to connect frontend forms to backend APIs using JavaScript fetch(). This activity improved my understanding of form validation and user interaction.

Backend Manager (John Roan Ballester)

I learned how backend routes process user requests and store data in MongoDB. Debugging API errors also improved my backend development skills.

Database Manager (Michelle Diaz)

This activity helped me understand how MongoDB stores user information through API requests. I also learned how to verify inserted documents in MongoDB Atlas.



GitHub Manager (Alyssa Jenille Reantaso)

I learned the importance of organizing commits and managing branches properly. Using GitHub improved our collaboration and version control skills.

Documentation Officer / Debugger (Ronel Garcia)

I learned how to properly document the workflow and organize project reports. Debugging frontend and backend issues also improved my problem-solving abilities.

